



Universidade Estadual de Mato Grosso do Sul

Ciência da Computação

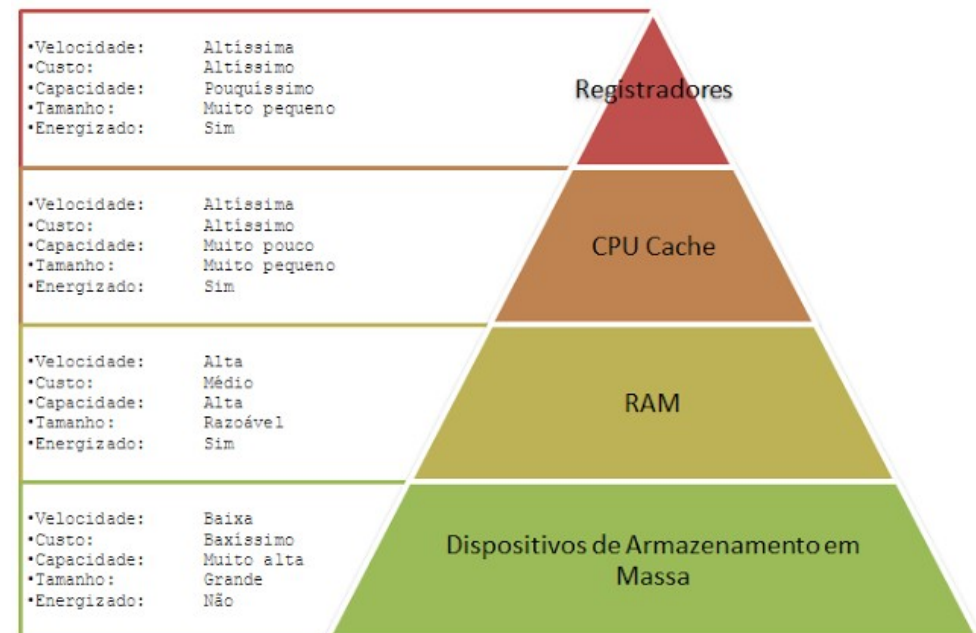
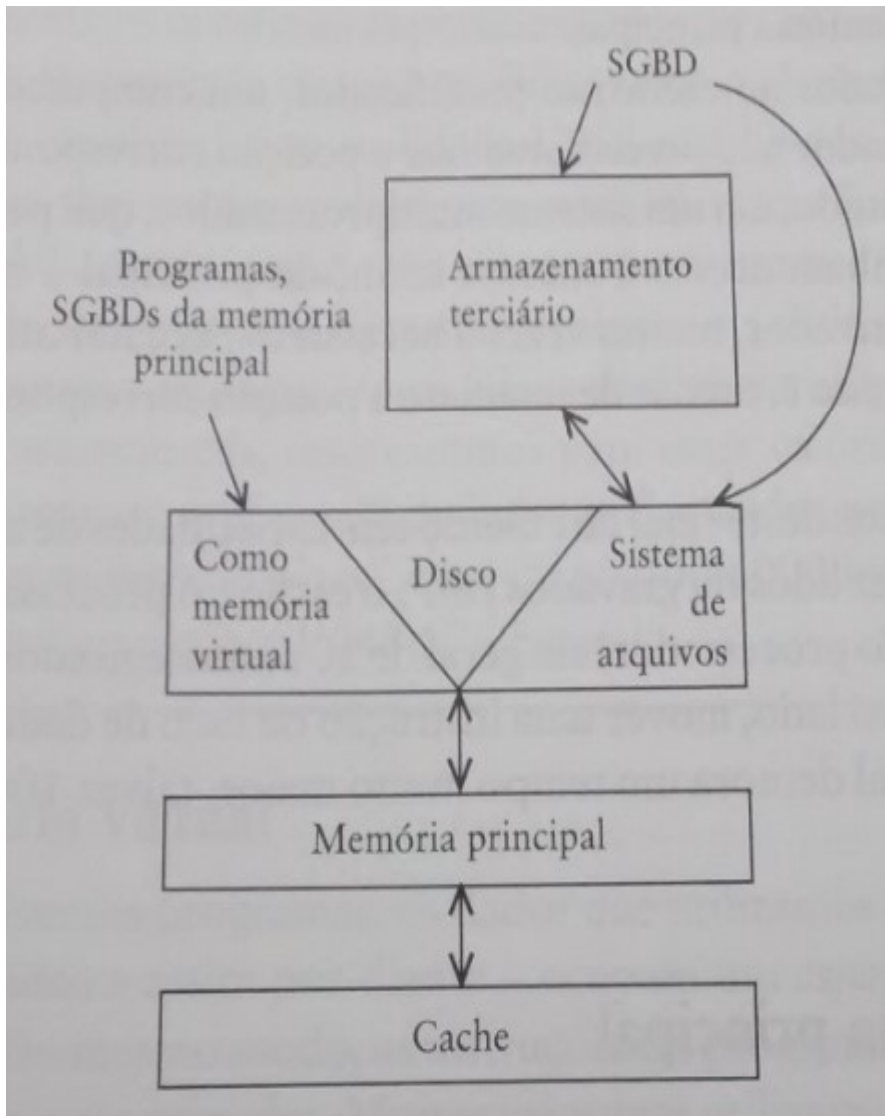
Banco de Dados

Organização de arquivos

Prof. Nilton
nilton@comp.uems.br

Hierarquia de Memória

- Tipos: primária, secundária e terciária
- Como é o acesso pela CPU?
- Como é a capacidade de armazenamento?
- KB, MB, GB, TB, PB
- Capacidade de armazenamento X velocidade X custo?



Hierarquia de Memória

→ Memória Principal

- Operada diretamente pela CPU
- DRAM (*Dynamic Random Access Memory*)
- Tudo que acontece no computador (execução de instruções e manipulações de dados) atuando sobre as informações residem na memória principal
- Acesso aleatório com acesso a qualquer posição no mesmo intervalo de tempo.

→ Cache

- É uma extensão da memória principal, conhecida como SRAM (*Static RAM*)
- Unidade de transferência é **bytes**
- Guarda pequenas instruções, dados e posições da memória principal
- Modificações são adiadas na memória principal
- Se não encontrar instruções ou dados busca na memória principal.

Exemplo Intel® Core™ i7-2670QM Processor
6 MB cache, até 32 GB RAM

Hierarquia de Memória

→ Memória Virtual

- Possibilita usar mais memória do que a fisicamente existente
- Usa o disco como cache para armazenamento secundário de dados
- Muito maior que a memória principal
- Páginas são descoladas entre o disco e a memória principal
- Uma página contém um conjunto de blocos
- Um disco é dividido logicamente em blocos
- Um bloco pode variar entre 512 bytes e 64KBytes.

Hierarquia de Memória

→ Memória Secundária

- O disco oferece suporte para memória virtual e sistemas de arquivos com acesso aleatório
 - Alguns **blocos** do disco armazenam (partes de) arquivos
 - Arquivos são movidos entre disco e memória principal em blocos, sob controle do SO ou SGBD
 - **Leitura de disco**: mover bloco do disco para a memória principal
 - **Gravação de disco**: mover bloco da memória principal para o disco
 - Leitura/gravação são operações de E/S de disco
 - Alguns blocos do disco armazenam páginas de memória virtual de um programa aplicativo
 - Disco pessoal de até 4 TBytes
- Uma fita usa o conceito de blocos mas o acesso aos dados é sequencial
- RAID (*Redundant Arrays of Inexpensive Disks*)
 - Vários discos independentes como um único disco lógico
 - Data striping (bit, bloco): E/S paralelo, Balanceamento carga, Redundância de dados⁵

Hierarquia de Memória

→ Memória Terciária

→ Bancos de dados podem crescer a tal ponto que uma coleção de unidades de discos de uma ou mais máquinas é insuficiente para o armazenamento

→ Tipos

→ Ad hoc em fita

- dados em rolos ou cassetes de fita armazenadas em prateleiras
- um humano localiza e monta a fita na unidade de leitura

→ Jukeboxes de discos ópticos

- prateleiras de discos de CD-ROM
- um braço robótico localiza e coloca o disco na unidade de leitura

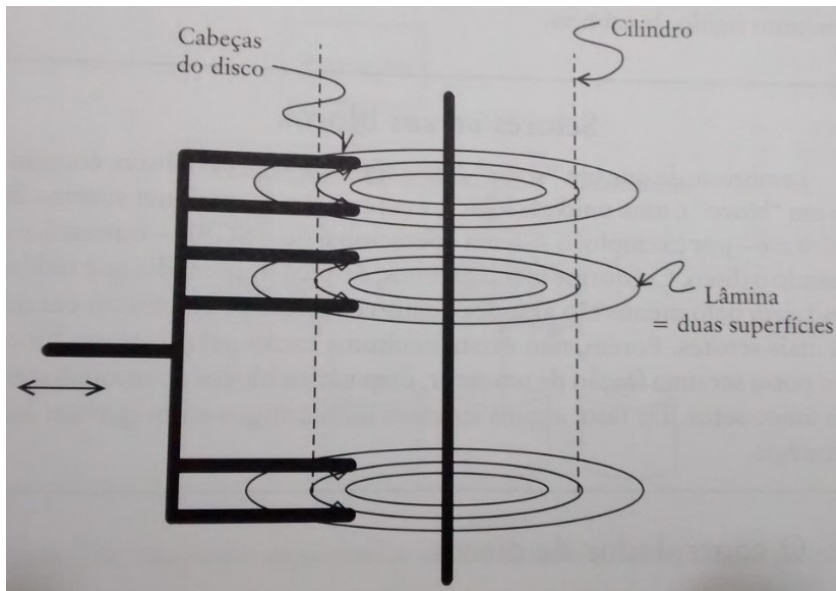
→ Silos de fita

- um de tamanho de uma sala que contém prateleiras de fitas
- um braço robótico localiza e coloca a fita na unidade de leitura

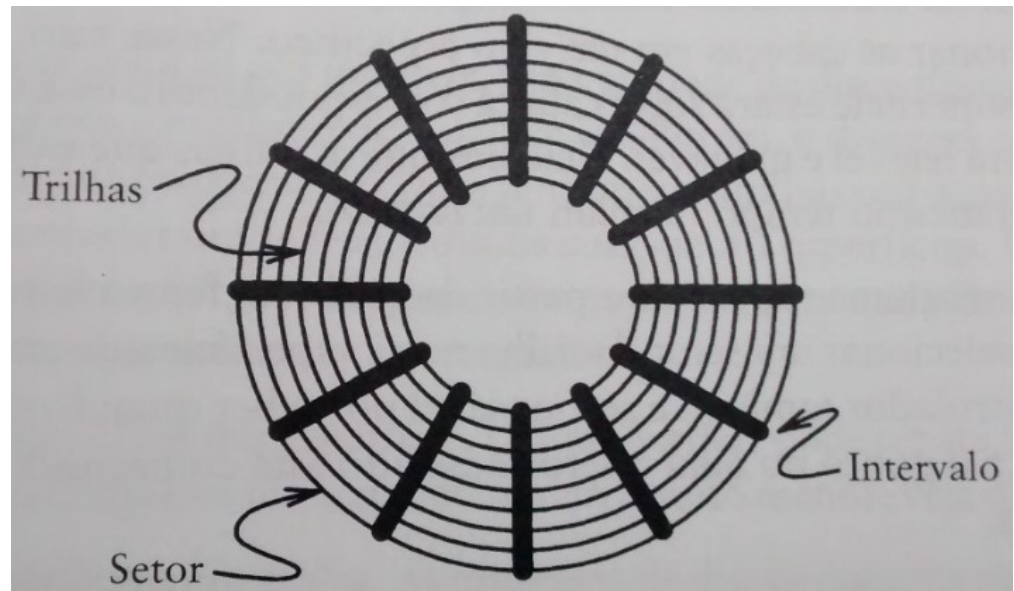
Armazenamento de Bancos de Dados

- Grandes volumes de dados
- Dados persistentes
 - persistem longos períodos de tempo
- Dados transientes
 - persistem durante a execução de uma aplicação (curto espaço)
- Armazenamento secundário
 - BD muito grande para caber na memória principal
 - perdas de dados são menos freqüentes
 - custo de armazenamento baixo

Discos e sua mecânica

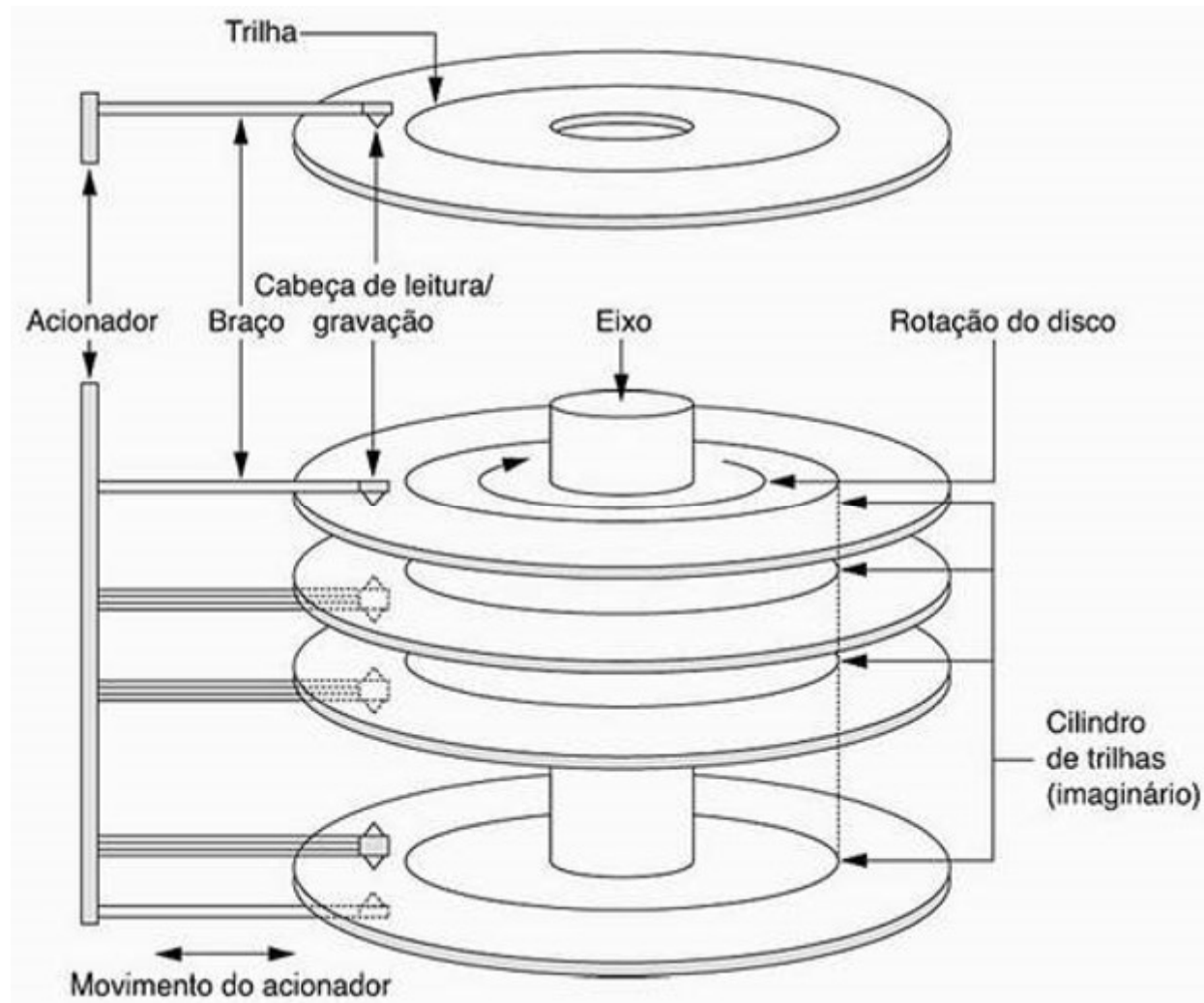


- Conjunto de discos/lâminas que giram ao redor de um eixo central
- Conjunto de cabeças de leitura/gravação
- A superfície superior e inferior da lâmina são recobertas por uma fina camada de material magnético onde os bits são armazenados
- Desktop (3.5 polegadas) ou Notebook (2.5 polegadas).

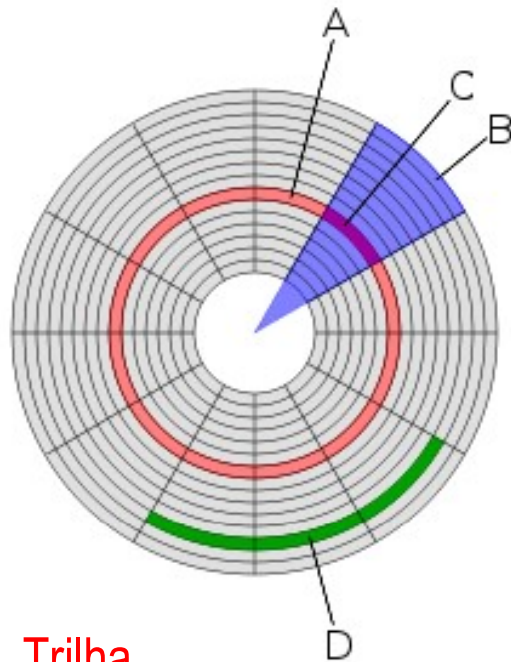


- Os bits estão armazenados nas trilhas (0 magnetizado em uma direção e 1 magnetizado em direção oposta)
- Bits agrupados formam uma informações (8bits=1byte)
- As trilhas estão organizadas em setores, que são segmentos do círculo separados por intervalos
- Um intervalo não são magnetizados
- Um setor é uma unidade indivisível, quando se refere a E/S em disco
- Um cilindro são trilhas de mesmo diâmetro nas diversas superfícies do disco.

Discos e sua mecânica



Unidade de alocação



- (A) Trilha
- (B) Setores (Setor Geométrico)
- (C) Setor de trilha
- (D) Unidades de alocação

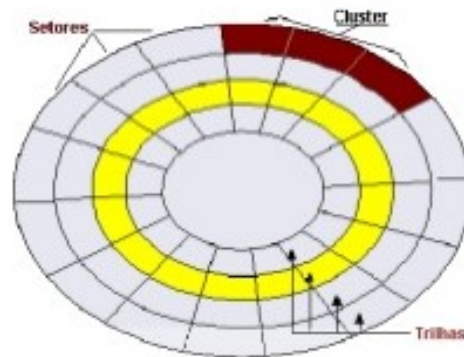
- Unidade de alocação ou *cluster* é a unidade básica de alocação do espaço em disco.
- Quando a unidade de alocação é definida?
- O sistema de arquivos aloca grupos contínuos de setores (*clusters* ou unidades de alocação) ao invés de setores individuais, para evitar overhead
- Um disco com setores de 512B
 - Para uma unidade de alocação de 512B contém 1 setor
 - Para uma unidade de alocação de 4KB contém 8 setores
- Uma unidade de alocação varia de 512B (1 setor) a 64 KB (128 setores)
- Quanto maior a unidade de alocação mais espaço¹⁰ será desperdiçado em disco para arquivos pequenos.

Unidade de alocação: exemplo

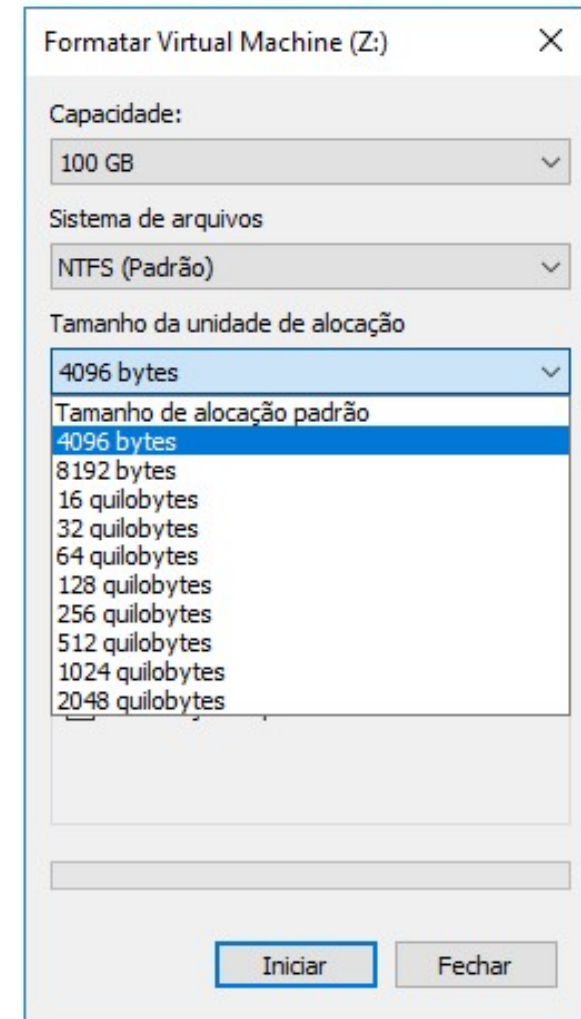
- A unidade de alocação varia entre 4096B à 2048KB
- Se o setor do disco for 512B
 - Teremos uma unidade de alocação entre 8 e 4096 setores

Clusters

- Após a formatação, o HD é dividido em clusters.
 - Cluster: conjunto de setores.
 - É a menor unidade de armazenamento que o **Sistema Operacional** pode acessar.



Setor 1	Setor 2	Setor 3	Setor 4	Setor 5	Setor 6	Setor 7	Setor 8	Setor 9	Setor 10	Setor 11	Setor 12
Cluster 1				Cluster 2				Cluster 3			

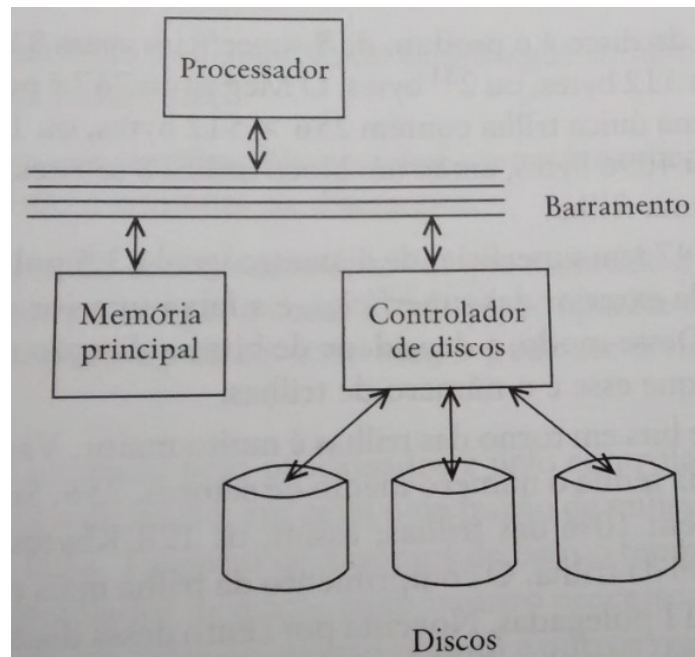


Setores versus blocos

- Setor é uma unidade física do disco
- Bloco é uma unidade lógica para acesso dos dados entre o disco e a memória principal
- Um bloco armazena um ou mais setores.

O controlador de discos

→ Um controlador de discos controla um ou mais dispositivos de discos



→ Funcionalidades

- Controlar o acionador mecânico que move o conjunto de cabeças para posicionar as cabeças em um raio específico. Nesse raio, uma trilha de cada superfície estará sob a cabeça correspondente. As trilhas sob as cabeças ao mesmo tempo formam um cilindro.
- Selecionar uma superfície e selecionar um setor da trilha
- Transferir os bits entre o setor desejado e a memória principal

Características do armazenamento de disco

- **Velocidade de uma rotação (ou latência) do conjunto de discos**
 - Os discos giram continuamente
 - Difícilmente os setores a serem lidos estarão sob a cabeça de leitura/gravação
 - 5400 RPM – uma rotação completa a cada 11 milissegundos ($60 / 5400 \times 1000$) aprox.
- **Número de lâminas por unidade (cerca de 5, então 10 superfícies)**
- **Número de trilhas por superfície (cerca de 10.000)**
- **Número de bytes por trilha**
 - 10^5 (100KBytes) ou mais bytes por trilha
 - Numa trilha até 500 setores por trilha
 - Um setor entre 512 e 4096 bytes

Características do armazenamento de disco

→ Um exemplo: disco Megatron

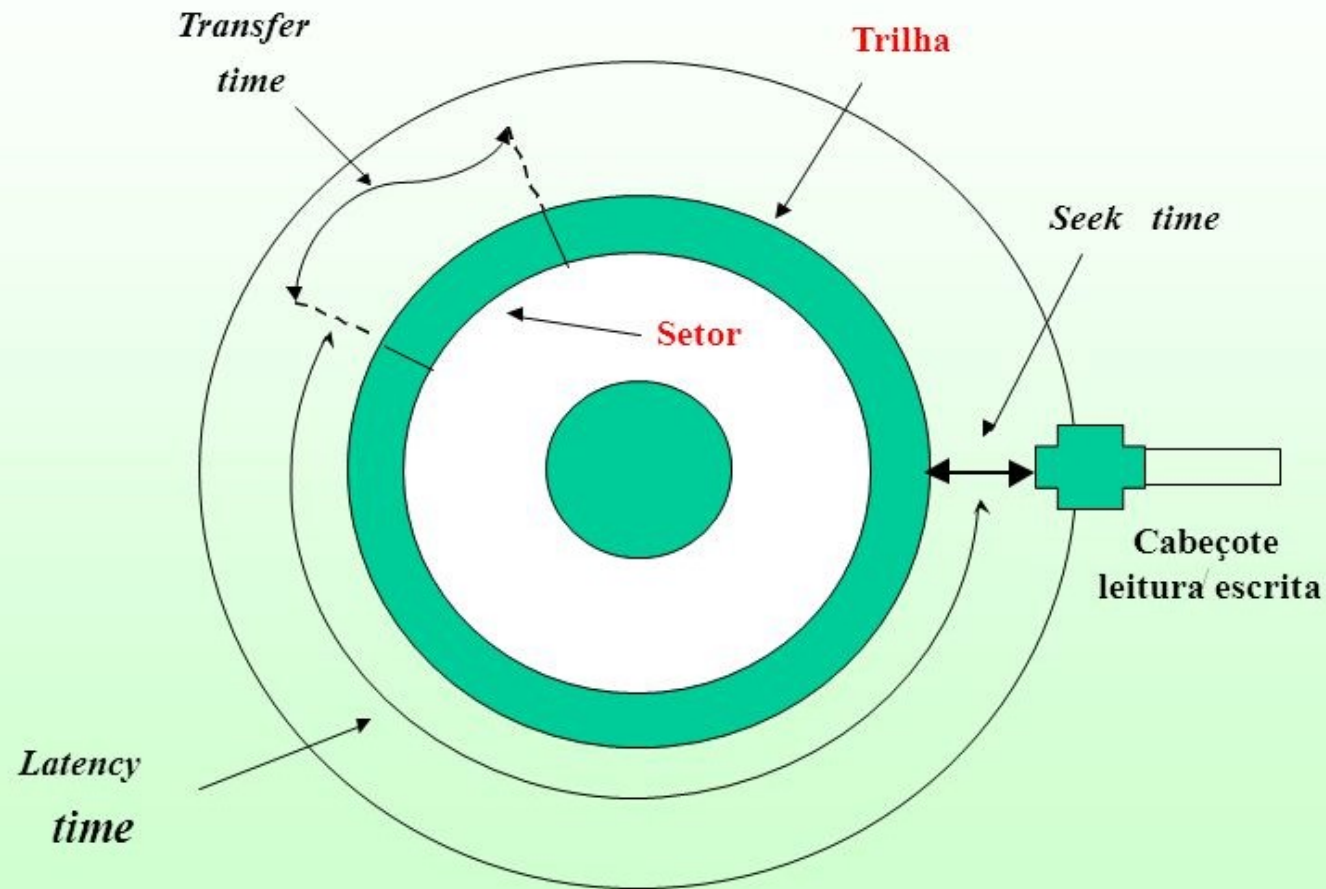
- Tem 4 lâminas fornecendo 8 superfícies
- Tem 8.192 trilhas por superfície
- Tem (em média) 256 setores por trilha
- Tem 512 bytes por setor

A capacidade do disco é $8 \times 8.192 \times 256 \times 512 = 8$ Gigabytes

Características de acesso ao disco

- Os blocos (ou setores consecutivos que formam os blocos) são lidos ou gravados quando:
 - a) As cabeças estão posicionadas no cilindro que contém a trilha na qual o bloco está localizado
 - b) Os setores que contém o bloco se deslocam sob a cabeça do disco à medida que todo o conjunto de disco gira
- Tempo de latência do disco: tempo entre momento em que é emitido o comando para ler um bloco e momento em que o conteúdo do bloco aparece na memória principal
 - **Tempo de busca (seek time)**: tempo para posicionar as cabeças no cilindro apropriado, sendo 0 ou distância a percorrer X (está entre 3X e 20X).
 - **Tempo de rotação (rotational delay)**: tempo para o disco girar até que a cabeça alcance o primeiro dos setores que contém o bloco.
 - **Tempo de transferência (block transfer time)**: cópia dos blocos
- O tempo de busca e de rotação costumam ter custo maior que o de transferência, mas se transferir vários blocos consecutivos na mesma trilha ou cilindro pode evitar aqueles custos, exceto para o 1º bloco
- A localização dos dados no disco é um **gargalo principal** em aplicações de bancos de dados.

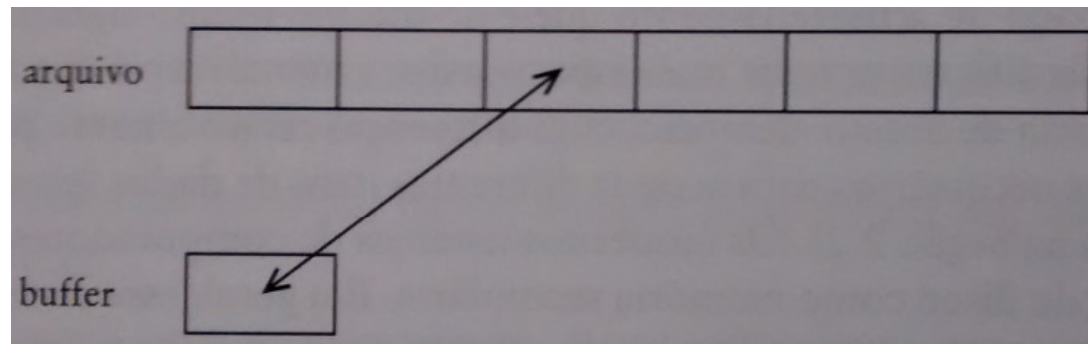
Temporização de acesso ao disco



$$t_{\text{acesso}} = t_{\text{seek}} + t_{\text{latência}} + t_{\text{trasnf.}}$$

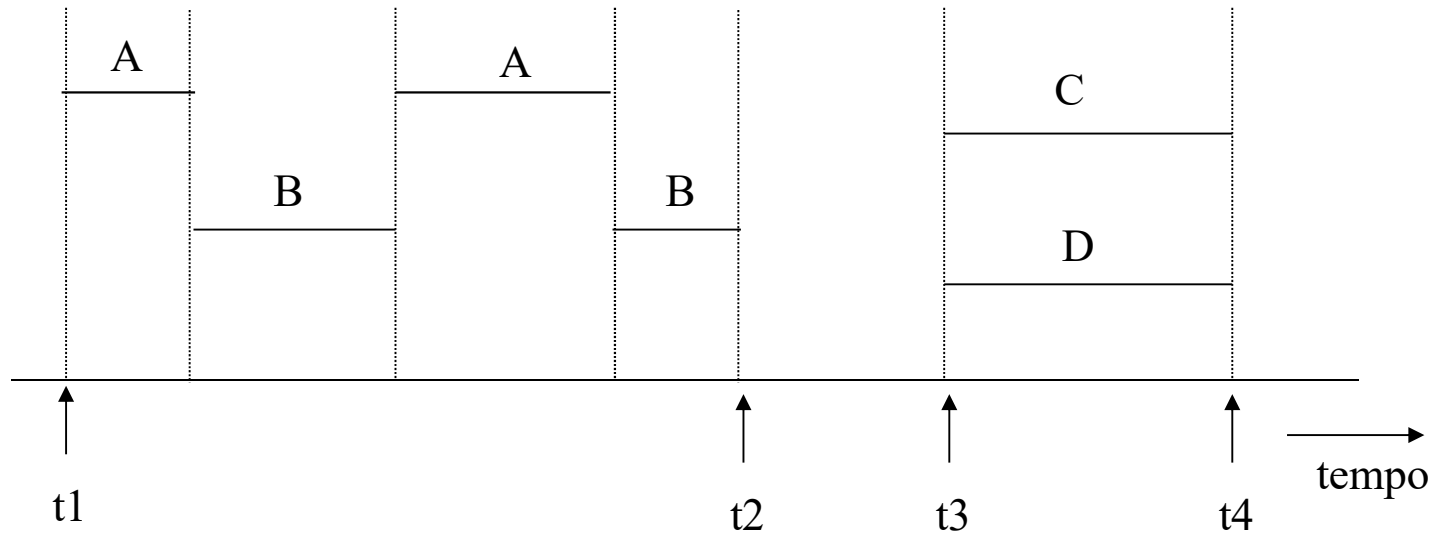
Bufferização de blocos

- Minimizar o gargalo de acesso ao disco
- Muitas páginas (blocos) de disco e poucas páginas de memória
- Tempo para processar um bloco na memória é menor do que o tempo exigido para ler o próximo bloco do disco



- Vários buffers na memória principal para aumentar a velocidade de transferência
- Enquanto um buffer está sendo lido/escrito um outro está sendo processado pela CPU
- Política de colocação de novas páginas no buffer
- Sem paralelismo verdadeiro: concorrência entrelaçada.

Concorrência Entrelaçada (t1 a t2) versus Paralela (t3 a t4)

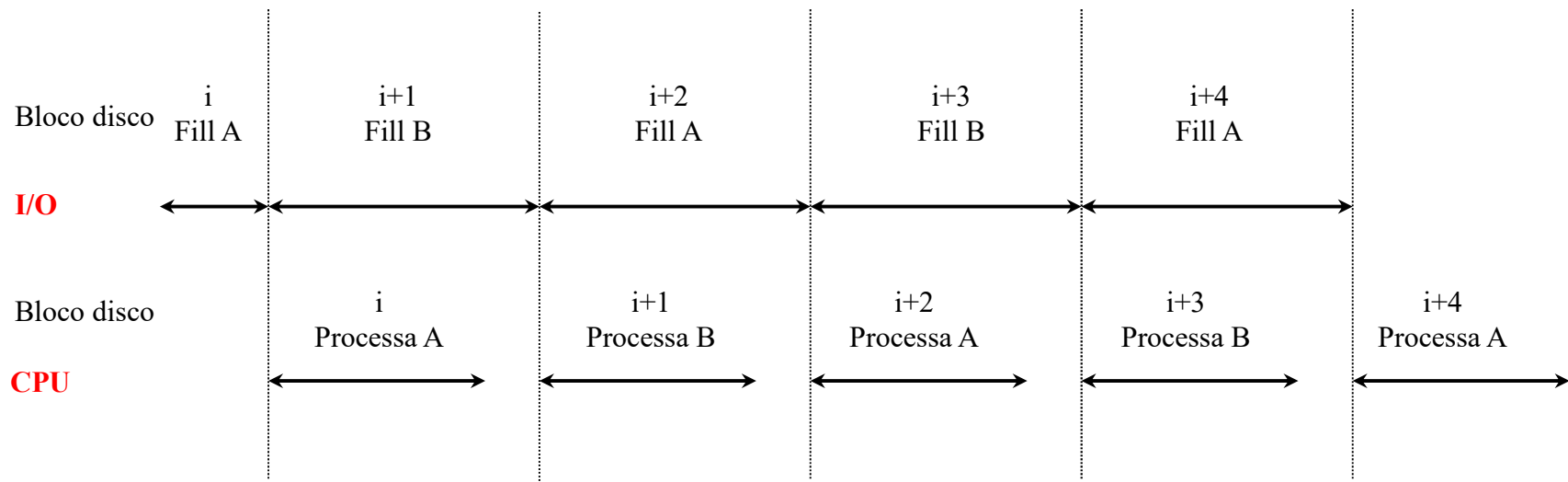


→ Paralelismo

- ◆ Múltiplos processadores CPUs
- ◆ Processador de E/S separado + 1 CPU (dupla bufferização)

Dupla Bufferização

- Técnica para minimizar o atraso em operações E/S
- Usa dois buffers (A e B) para, por exemplo, leitura do disco



- A CPU inicia o processamento de um bloco após sua leitura para memória principal
 - Ao mesmo tempo, o processador E/S lê o próximo bloco do disco para um buffer diferente
- tempo

- A técnica permite a leitura/escrita contínua de blocos de disco consecutivos, eliminando o tempo de busca e de rotação, menos para o 1º bloco
- Os dados prontos pra processamento, reduzo tempo de espera dos programas.

Gerenciador de Buffer

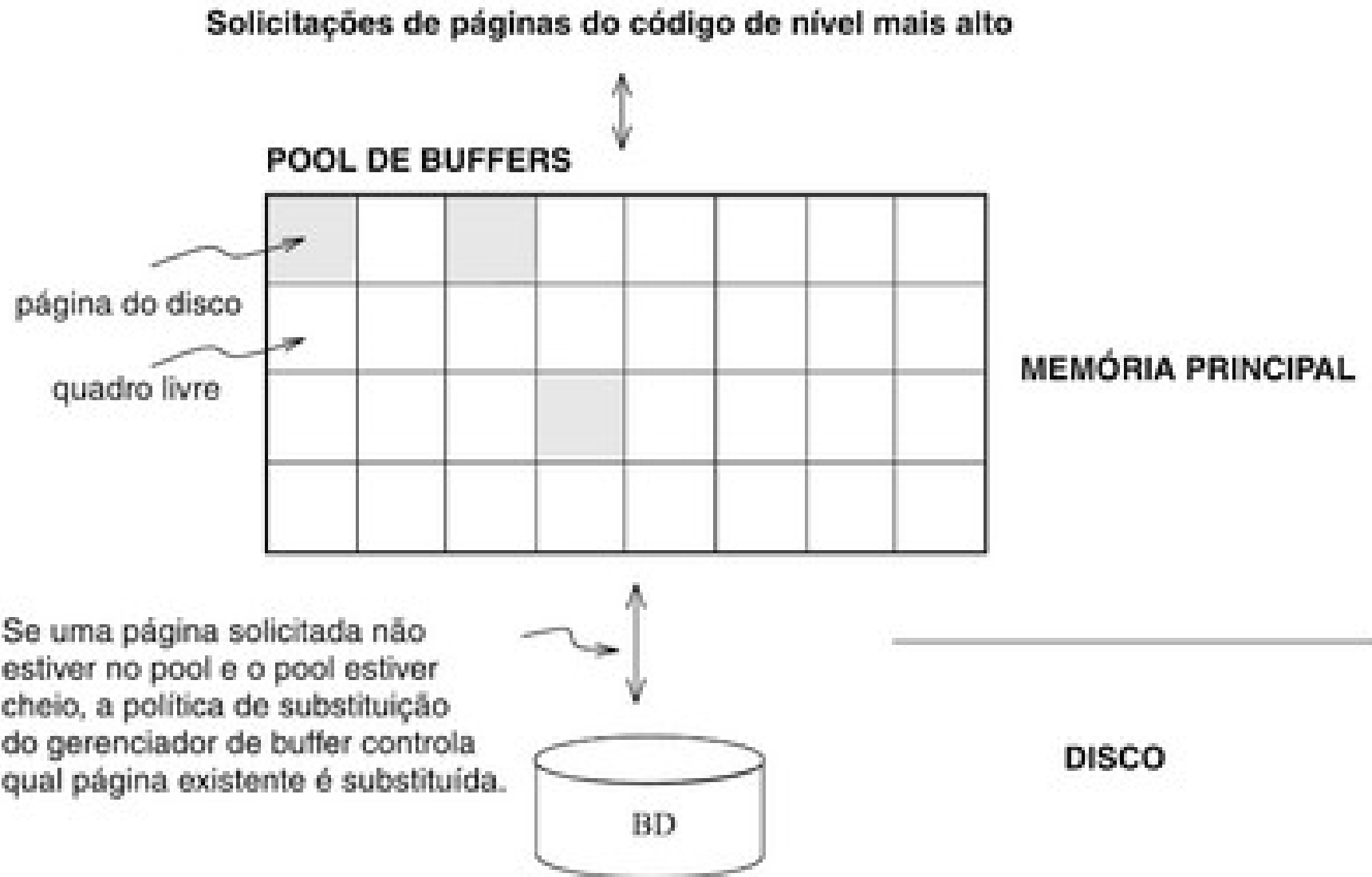


Figura 9.3 O pool de buffers.

Gerenciador de Buffer

→ Interno, transparente para os níveis mais altos

- ◆ Contador de usuários da página (bloco):
- ◆ Se a página sofreu atualização desde que foi levada para o buffer

→ Na requisição de uma página de disco

- ◆ Verifica se ela já está na área de buffer
- ◆ Se não está
 1. escolhe um quadro para substituição de acordo com a política
 2. se houve atualização do quadro antigo, levá-lo para o disco
 3. lê a página requisitada no quadro escolhido

→ O quadro só pode ser substituído depois que foi liberado

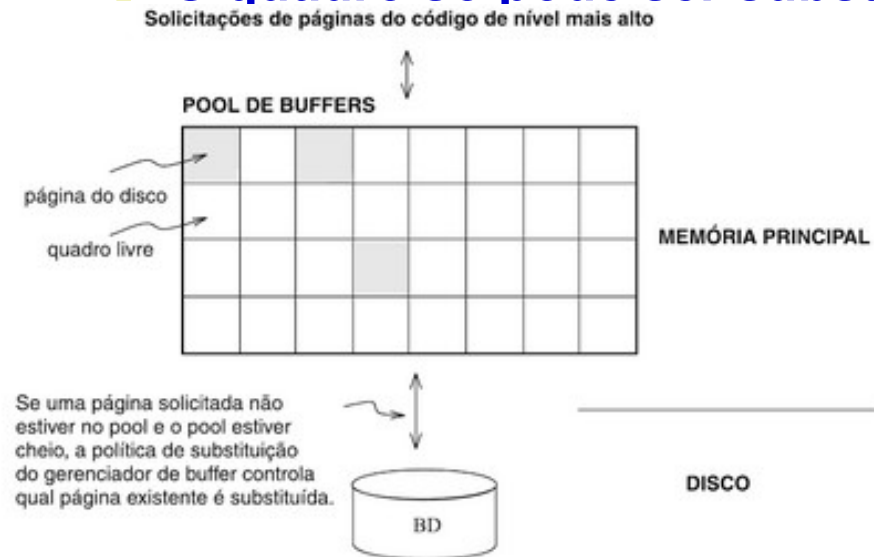


Figura 9.3 O pool de buffers.

→ Para cada quadro duas variáveis

- $pin_count = 0$ (nenhum usuário corrente)
- $pin_count > 0$ (um ou mais usuários correntes)
- $dirty = 0$ (página sem modificação)
- $dirty = 1$ (página modificada e precisa ser gravada em disco antes liberar quadro)

Políticas de substituição de página no buffer

→ LRU (*Least Recently Used*)

- substitui a que foi menos recentemente usada
- Implementado como uma fila de ponteiros para quadros com *pin_count* = 0

→ por relógio (variante do LRU)

- Ao invés de tirar a página que é intensamente usada do buffer é dada uma segunda chance a ela, deixando-a na fila CIRCULAR

→ MRU (*Most Recently Used*)

- substitui a que foi mais recentemente usada
- Implementado como uma pilha de ponteiros para quadros com *pin_count* = 0

→ LFU (*Least Frequently Used*)

- substitui a com menor número de acessos

→ FIFO (*First in First out*)

→ randômica

Guardando os Registros do Arquivo no Disco

→ Arquivos

→ Registros

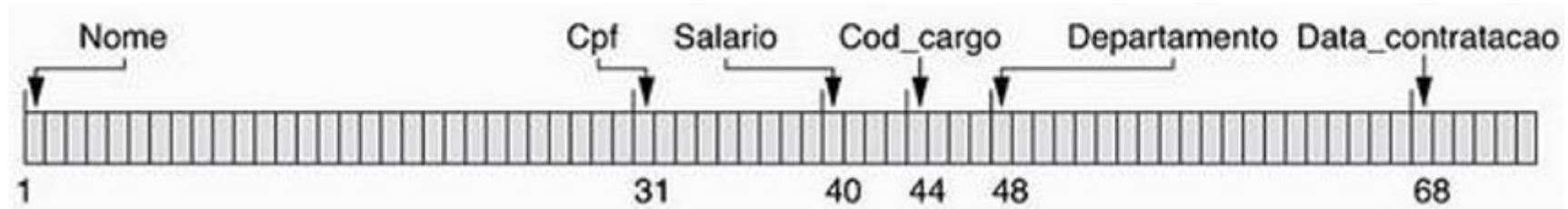
◆ Tamanho fixo e variável

→ Bloqueamento de Registros

→ Alocação de Blocos de Arquivos

Registros

→ Registro de tamanho fixo



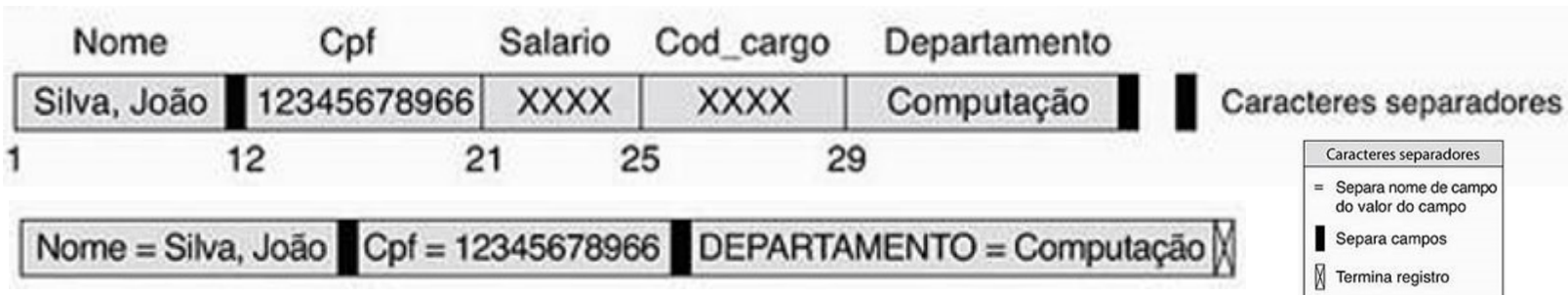
→ Registro de tamanho variável

→ Registros são do mesmo tipo

- Um ou mais campos pode ter tamanho variável
- Um ou mais campos pode ter múltiplos valores
- Um ou mais campos são opcionais

→ Registros são de tipos diferentes

- Agregar informações diferentes de uma entidade

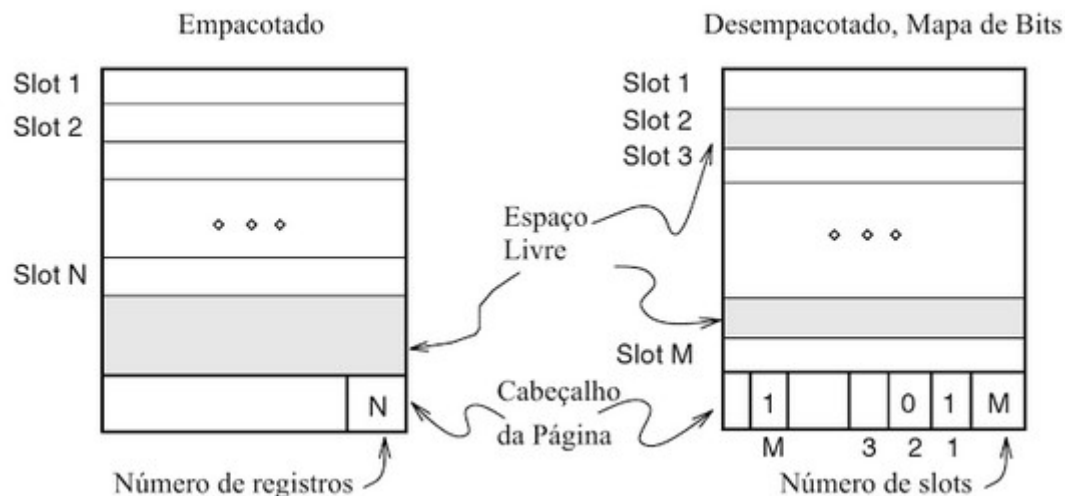


Formatos de página

- abstração de página é apropriada em questões de E/S
- vários níveis mais altos do SGBD vêem os dados como uma coleção de registros
- cada página tem vários slots
- em cada slot um registro
- identificador de registro (rid)
 - ◆ <identificador da página, número slot>
- registros de comprimento fixo
 - ◆ por cálculo de offset
- registros de comprimento variável
 - ◆ diretório de slots da página
 - ◆ <offset do registro, tamanho do registro>

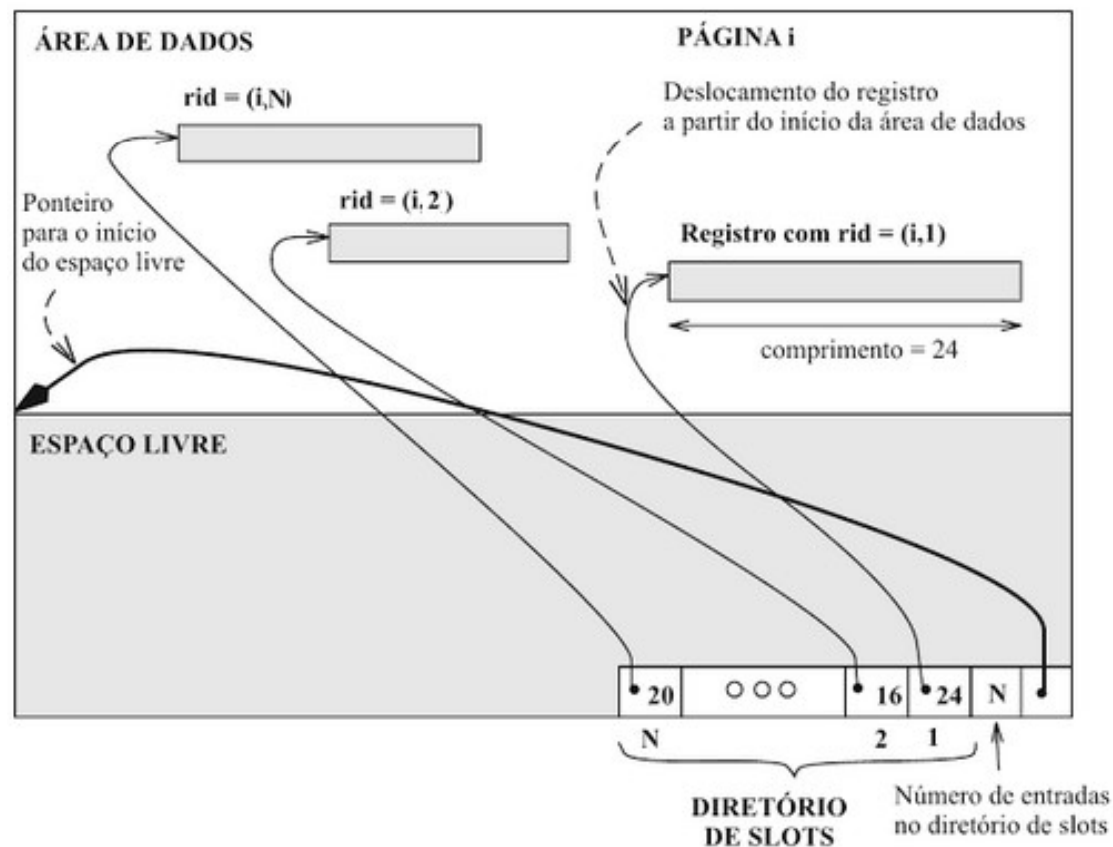
Formatos de página: registros de comprimento fixo

- quantidade fixa de slots
- registros organizados consecutivamente
- técnicas de organização na página quando incluir um novo registro ou excluir um registro existente
 - **Empacotamento**: manter os slots vazios no final da página; acesso direto ao espaço livre por cálculo de offset;
 - **Mapa bits**: se slot está ocupado (1) e desocupado (0); acesso ao espaço livre via espaço de diretório



Formatos de página: registros de comprimento variável

- não tem como manter uma quantidade fixa de slots
- procurar um slot de tamanho adequado para o registro
- Ideal manter espaços vazios contíguos
- técnica de organização em **diretório de slots**
 - <deslocamento registro, comprimento registro>



Blocamento de Registros

- **Bloco - Unidade de transferência de dados entre disco e memória**
- **Organização não espalhada**
- **Organização espalhada**

Blocamento de Registros

→ Organização não espalhada

- ◆ registros não são espalhados em mais que um bloco (página)
- ◆ útil para registros pequenos



📄 **B**: tamanho bloco (bytes)

📄 **R**: tamanho registro (bytes)

📄 **fator de bloco** (n registros num bloco) $fb = \text{chão}(B / R)$ para $B \geq R$

📄 **espaço vazio**

$$V = B - R * fb$$

Blocamento de Registros

→ Organização espalhada

- ◆ aproveita todo o espaço do bloco (página)
- ◆ útil para registros grandes



→ O número de blocos necessários para um arquivos com r registros:

📄 $b = \text{Teto}(r / fb)$, onde r é o número de registros

Blocamento de Registros - exemplo

- ◆ Um arquivo com 10.000 registros está armazenado usando organização não espalhada em blocos de 512 bytes. Cada bloco contém registros de 50 bytes. Determine fb , b e V .

📄 $fb = \text{chão}(B/R)$

📄 $fb = \text{chão}(512/50)$

📄 $fb = 10 \text{ registros}$

📄 $b = \text{Teto}(r/fb)$

📄 $b = \text{Teto}(10.000/10)$

📄 $b = 1.000 \text{ blocos}$

📄 $V = B - R * fb$

📄 $V = 512 - 50 * 10$

📄 $V = 12 \text{ bytes}$

Alocação de Blocos de Arquivos no Disco

→ contígua

- ◆ um arquivo ocupa um conjunto de blocos (ou setores) consecutivos no disco
- ◆ a localização de um arquivo é através do número do setor inicial e o tamanho do arquivo - em blocos
- ◆ o acesso seqüencial é através do número do último bloco mais o incremento
- ◆ o acesso direto ao bloco i que inicia no bloco b é o acesso do bloco $b+i$
- ◆ difícil expandir o arquivo, ou seja, encontrar um espaço livre que ocupa n blocos contíguos.

→ ligada

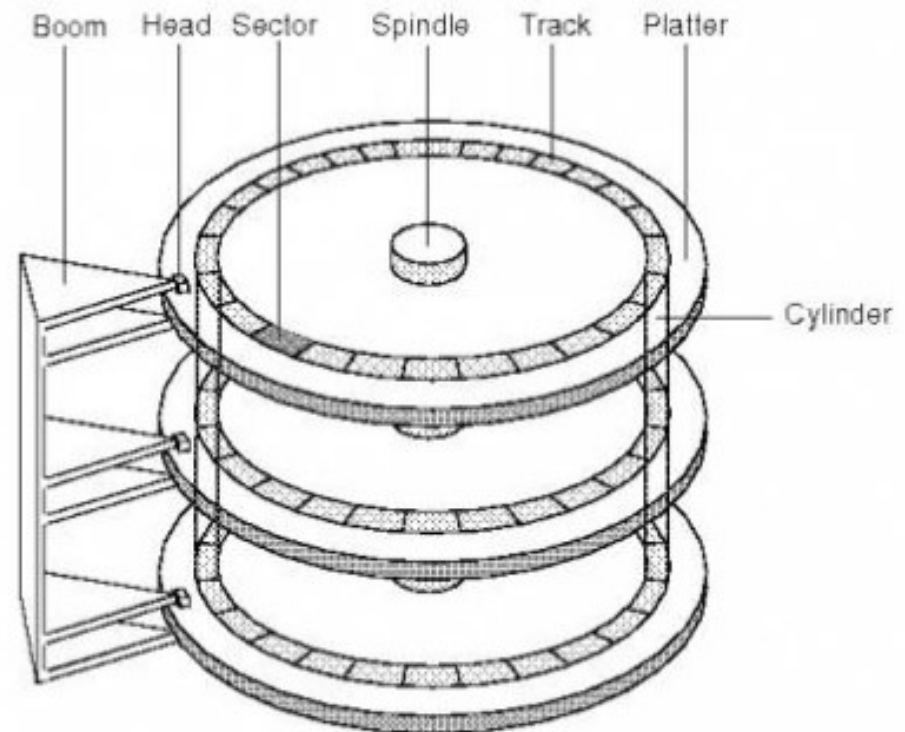
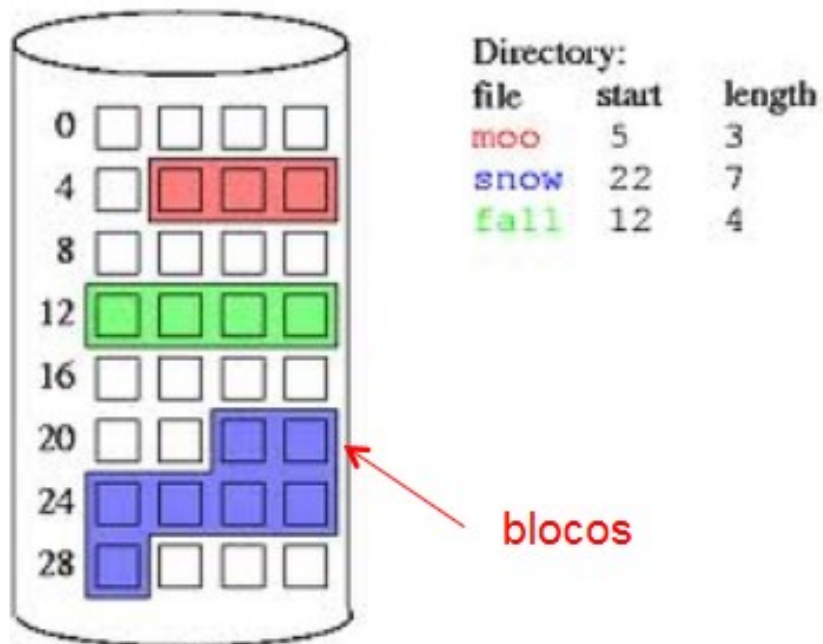
- ◆ um arquivo corresponde a uma lista encadeada de blocos, estando eles em qualquer local do disco
- ◆ um arquivo pode ser criado com tamanho zero e pode crescer até o limite do disco
- ◆ acesso apenas seqüencial.

→ indexada

- ◆ um arquivo é acessado através de uma tabela de índice, onde cada entrada dessa tabela contém o endereço de um dos blocos físicos do arquivo
- ◆ permite resolver o problema do crescimento de um arquivo e seu acesso direto.

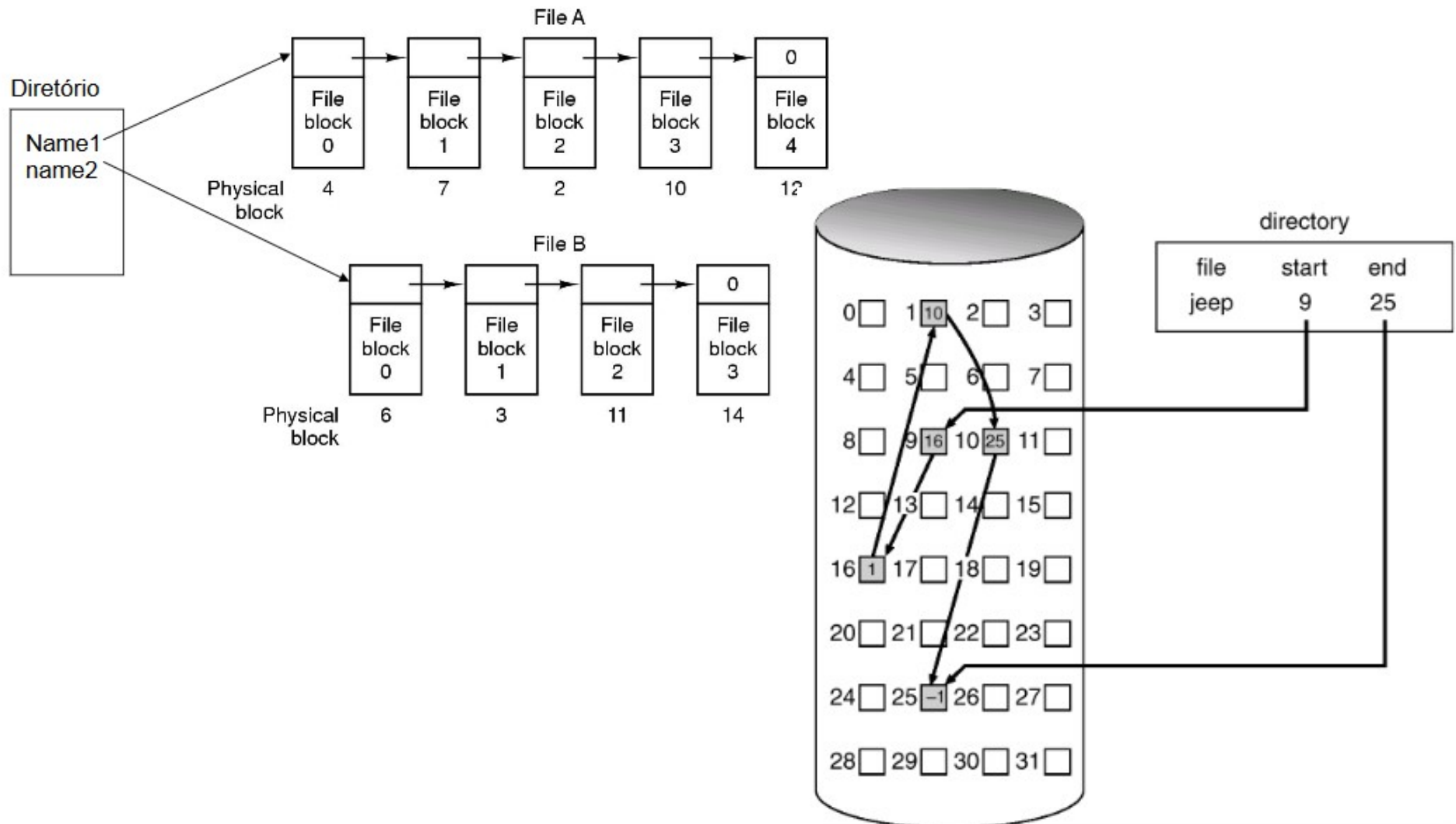
Arquivo com blocos contíguos

- Um arquivo é composto de uma seqüência de blocos de mesmo tamanho (#bytes)
- O disco é um vetor de blocos, cada um com um endereço único (endereço lógico do bloco)
- O endereço lógico é traduzido para (cilindro, trilha, setor)



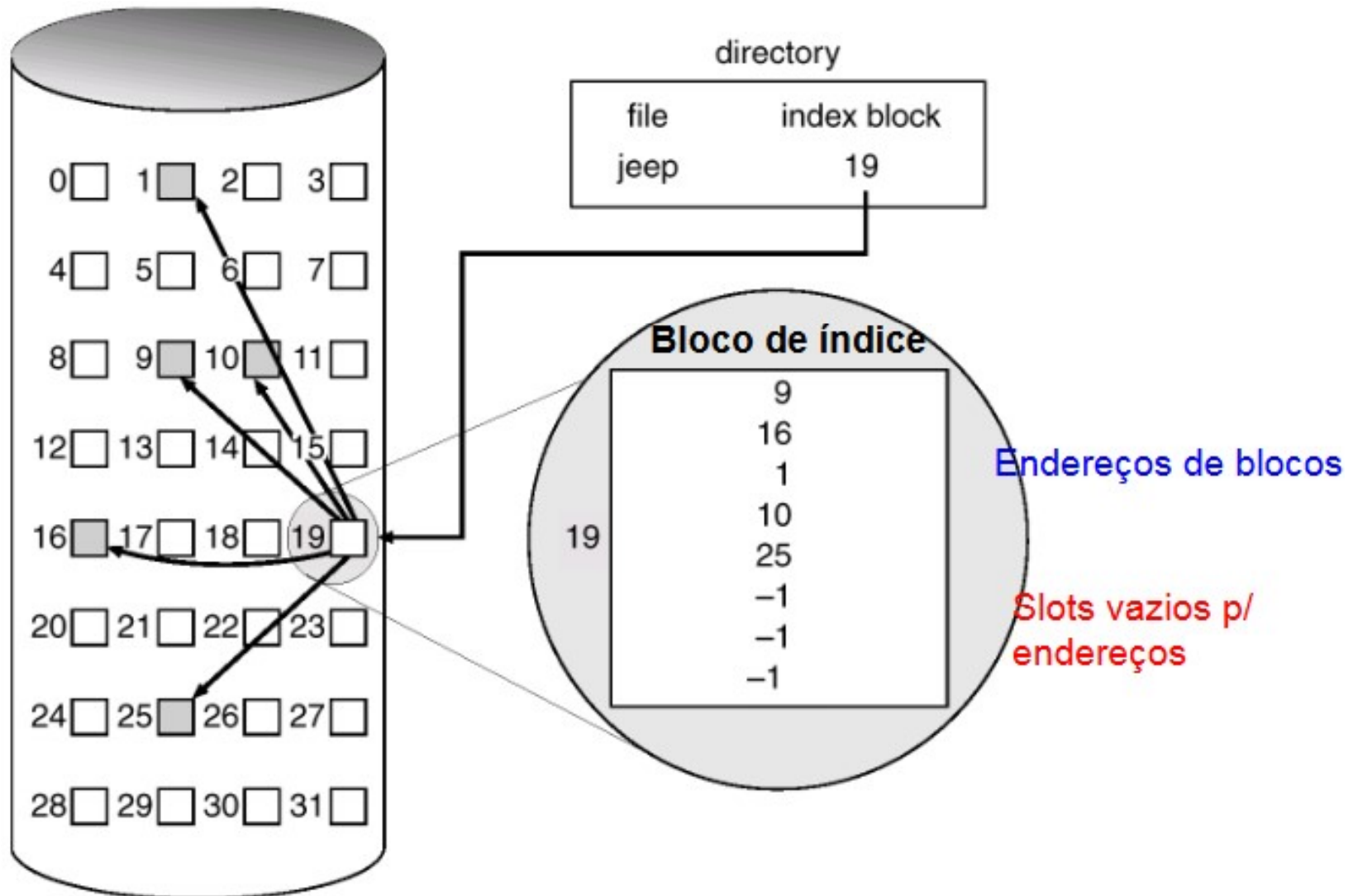
Arquivo com blocos ligados

→ Primeiros bytes de cada bloco contém um ponteiro (endereço de disco) para o próximo bloco da lista



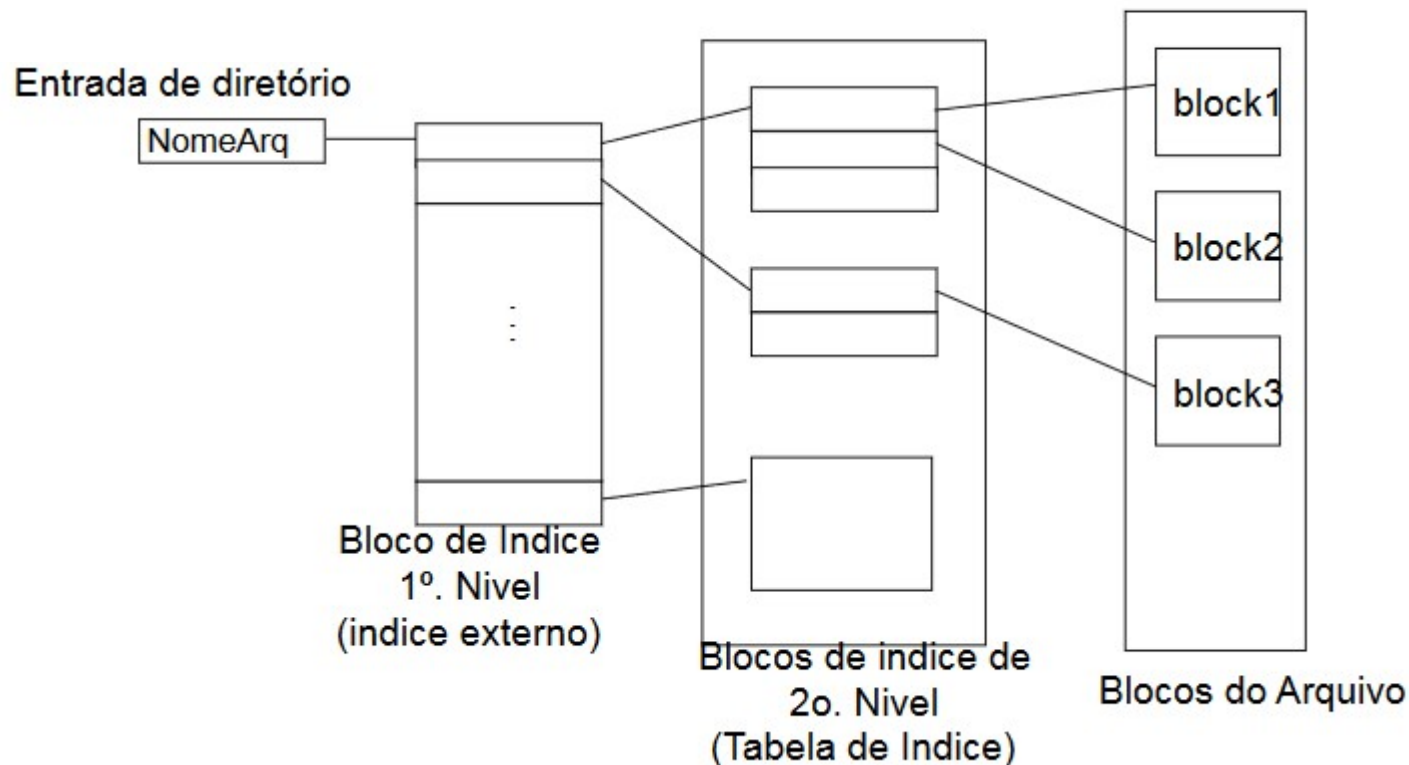
Arquivo com blocos indexados

→ Todos os endereços de blocos são mantidos em um bloco de índice



Arquivo com blocos indexados multinível

- Para um arquivo muito grande o bloco de índice pode não comportar as entradas para o arquivo
- A entrada de diretório aponta para o bloco do índice de 1º nível
- O endereço físico do registro (byte) no bloco precisa ser calculado



Operações em arquivos

→ Busca e Modificação

→ Modificação

◆ inserção, remoção e atualização de campos

→ Critério de busca

→ Registro corrente

Organização de arquivos e Método de acesso

- Uma **organização de arquivos** refere-se à organização dos dados de um arquivo em registros, blocos e estruturas de acesso, distribuição e interligação no disco
- Um **método de acesso** é grupo de programas que permitem operações
- Diferentes métodos de acesso a uma organização de arquivos.

Organização **primária** de arquivos

1 - Arquivos de registros não ordenados

2 - Arquivos de registros ordenados

3 - Técnicas hashing

1 - Arquivos de Registros Não Ordenados

→ Busca sequencial

→ na média a busca em blocos/2 ou pior caso todos blocos

→ Inserção direta no final do arquivo

→ último bloco de disco do arquivo é copiado para o buffer;

→ registro é acrescentado no bloco e o bloco regravado no disco.

→ Remoção (2 técnicas)

→ 1 - lê o bloco, localiza o registro, remove o registro, grava o bloco

→ 2 - marcador de remoção;

→ nas duas técnicas precisa-se de reorganização periódica

→ Criação de uma cópia ordenada do arquivo

→ Registro de tamanho fixo com blocos não espalhados e alocação contígua – acesso direto ou arquivo relativo

→ Se registros no arquivo forem numerados 0, 1, 2, ..., r-1 e

→ E registros no bloco 0, 1, 2, ..., fb-1

→ i-ésimo registro no arquivo esta localizado no bloco $\text{chão}(i / fb)$

→ $i \bmod fb$ registro nesse bloco;

2 - Arquivos de Registros Ordenados

- Os registros do arquivo são ordenados fisicamente com base em um dos seus campos
- Campo de ordenação do arquivo
 - bom que seja chave
- O acesso ao arquivo: seqüencial ou ordenado.

2 - Arquivos de Registros Ordenados

	Nome	Cpf	Data_nascimento	Cargo	Salario	Sexo
Bloco 1	Aaron, Eduardo					
	Abílio, Diana					
	⋮					
	Acosta, Marcos					
Bloco 2	Adams, João					
	Adams, Roberto					
	⋮					
	Akers, Janete					
Bloco 3	Alexandre, Eduardo					
	Alfredo, Roberto					
	⋮					
	Allen, Samuel					
⋮						
Bloco $n-1$	Wanderley, Jaime					
	Wesley, Ronaldo					
	⋮					
	Wong, Manuel					
Bloco n	Wong, Pâmela					
	Wuang, Charles					
	⋮					
	Zimmer, André					

2 - Arquivos de Registros Ordenados

→ Busca eficiente

- ◆ nenhuma ordenação é necessária
- ◆ o próximo registro pode estar no mesmo bloco
- ◆ busca binária a nível de bloco: $\log_2(b)$ blocos

◆ Para a busca binária

- ◆ se os blocos forem numerados com 1, 2, ..., b blocos
- ◆ os registros estão ordenados crescentemente por seu campo-chave de ordenação
- ◆ K é o valor de pesquisa pelo campo-chave de ordenação
- ◆ se os endereços de disco dos blocos de arquivo estejam disponíveis no cabeçalho do arquivo
- ◆ o algoritmo a seguir pode ser utilizado

2 - Arquivos de Registros Ordenados

→ Busca binária em uma chave de ordenação (K) de um arquivo de disco

```
 $l \leftarrow 1; u \leftarrow b; (* b \text{ é o número de blocos de arquivo } *)$   
enquanto  $(u \geq l)$  faça  
    início  $i \leftarrow (l + u) \text{ div } 2;$   
    leia bloco  $i$  do arquivo para o buffer;  
    se  $K < (\text{valor do campo-chave de ordenação do primeiro registro no bloco } i)$   
        então  $u \leftarrow i - 1$   
    senão se  $K > (\text{valor do campo-chave de ordenação do último registro no bloco } i)$   
        então  $l \leftarrow i + 1$   
    senão se o registro com valor do campo-chave de ordenação =  $K$  está no buffer  
        então vai para found  
    senão vai para notfound;  
fim;
```

→ Critérios de buscas em $>$, $<$, $\geq$, $\leq$ é ainda muito eficiente em arquivos de registros ordenados

```

 $l \leftarrow 1; u \leftarrow b; (*\text{ b é o número de blocos de arquivo }*)$ 
enquanto  $(u \geq l)$  faça
    início  $i \leftarrow (l + u) \text{ div } 2;$ 
    leia bloco  $i$  do arquivo para o buffer;
    se  $K < (\text{valor do campo-chave de ordenação do primeiro registro no bloco } i)$ 
        então  $u \leftarrow i - 1$ 
    senão se  $K > (\text{valor do campo-chave de ordenação do último registro no bloco } i)$ 
        então  $l \leftarrow i + 1$ 
    senão se o registro com valor do campo-chave de ordenação =  $K$  está no buffer
        então vai para found
    senão vai para notfound;
fim;

```

Bloco 1

Nome	Cpf	Data_nascimento	Cargo	Salario	Sexo
Aaron, Eduardo					
Abílio, Diana					
⋮					
Acosta, Marcos					

Bloco 2

Adams, João					
Adams, Roberto					
⋮					
Akers, Janete					

Bloco 3

Alexandre, Eduardo					
Alfredo, Roberto					
⋮					
Allen, Samuel					

2 - Arquivos de Registros Ordenados

→ Inserção e remoção são operações caras

→ registros precisam permanecer ordenados

→ Inserção

→ determinar a posição de inserção

→ criar espaço no arquivo para inserir o registro nessa posição

→ em arquivos grandes metade dos blocos precisam ser lidos/gravados e conseqüentemente os registros nesses blocos movimentados

→ Remoção

◆ menos custosa com uso de marcação e reorganização periódica

→ Como melhorar a inserção?

◆ manter algum espaço não usado no bloco; se encher problema surge

◆ arquivo mestre e de transação

◆ Novos registros, ordenação, mesclagem, custo de pesquisa.

2 - Arquivos de Registros Ordenados

→ Alteração de um campo não de ordenação

→ Alteração de um campo de ordenação

→ Exclusão seguida de inserção

3 – Técnicas Hashing

- *Hashing* é uma outra técnica de organização de arquivos primária com acesso muito rápido a registros
- Organização direta: arquivo de *hash* sobre campo de *hash*
- Condição de igualdade como campo *hash* do arquivo (em geral, campo chave)
- h = função de hash
 - gera o endereço do bloco de disco em que o registro está armazenado
- *Hashing*
 - interno (pesquisa)
 - externo (armazenamento em disco)

Hashing Interno - pesquisa

→ Implementa-se hashing usando vetores

→ 1) Vetor de índices variando de 0 até M-1

- ◆ Função de hash comum: $h(k) = k \bmod M$

→ 2) Uma implementação comum

- ◆ espaço de endereços : de 0 até M-1

- ◆ espaço de overflow : desde o endereço M indo até M+K

→ Tratamento de Colisões

→ 1) Hashing + lista ligada

→ 2) Endereçamento aberto

- ◆ procurar células vazias após endereço calculado

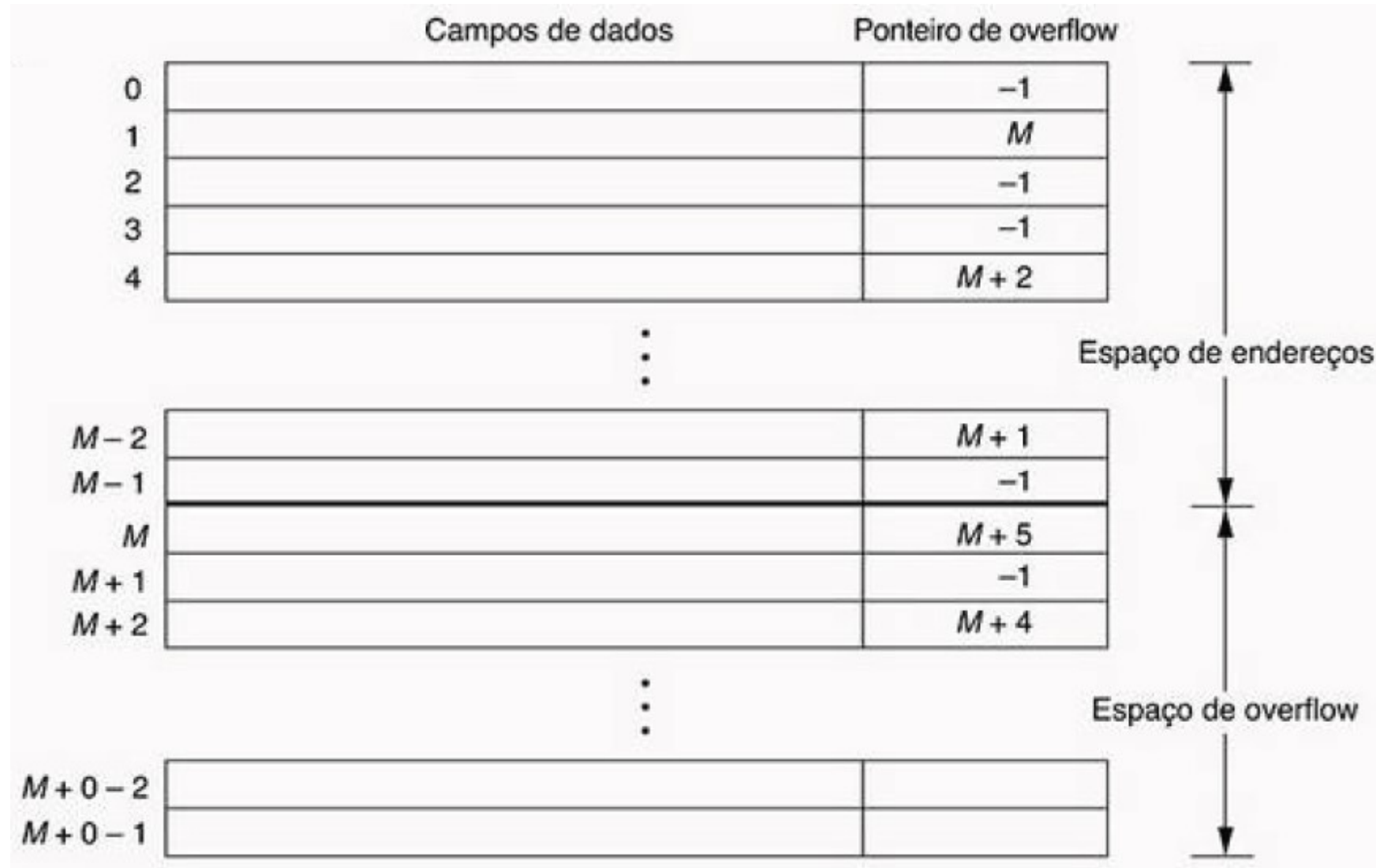
- ◆ espaço de endereços + espaço de overflow

→ 3) Encadeamento

- ◆ hashing múltiplo: várias funções de hashing

- 📄 se a função de hashing origina colisão, aplicar uma outra função de hashing até que não origine colisão

Hashing Interno - pesquisa



Ponteiro de overflow

-1 nenhuma colisão;

<> -1 tratamento de colisão usando lista ligada

Hashing Interno - pesquisa

método de Horner

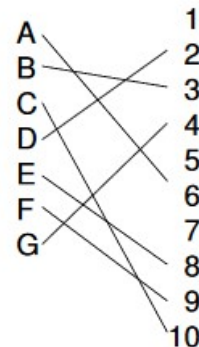
```
int hash(string v, int M) {  
    int i, h = v[0];  
    for (i = 1; v[i] != '\0'; i++)  
        h = h * 256 + v[i];  
    return h % M;  
}
```

→ Uma boa função *hash* deve

- ❑ Distribuir de forma uniforme as chaves de modo que minimize colisões
- ❑ A escolha do espaço de armazenamento deve estar entre 0,7 e 0,9 (NrChaves / NrLocais)
- ❑ Para NrLocais: escolher um número primo ou potência de 2

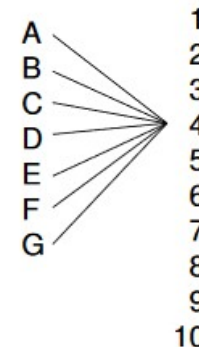
(a) melhor caso

registro endereço



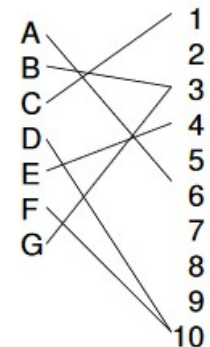
(b) pior caso

registro endereço



(c) caso aceitável

registro endereço



Hashing Externo – armazenamento disco

→ Para arquivos de disco.

- ◆ Para acompanhar as características de armazenamento em disco o espaço de endereço destino é composto de agrupamentos (*buckets*)
- ◆ Um agrupamento é composto de um ou mais blocos contíguos
- ◆ O problema da colisão é menos sério, pois os registros podem ser distribuídos dentro do agrupamento

→ A função de *hashing* mapeia a chave para um número de agrupamento relativo ao invés de absoluto

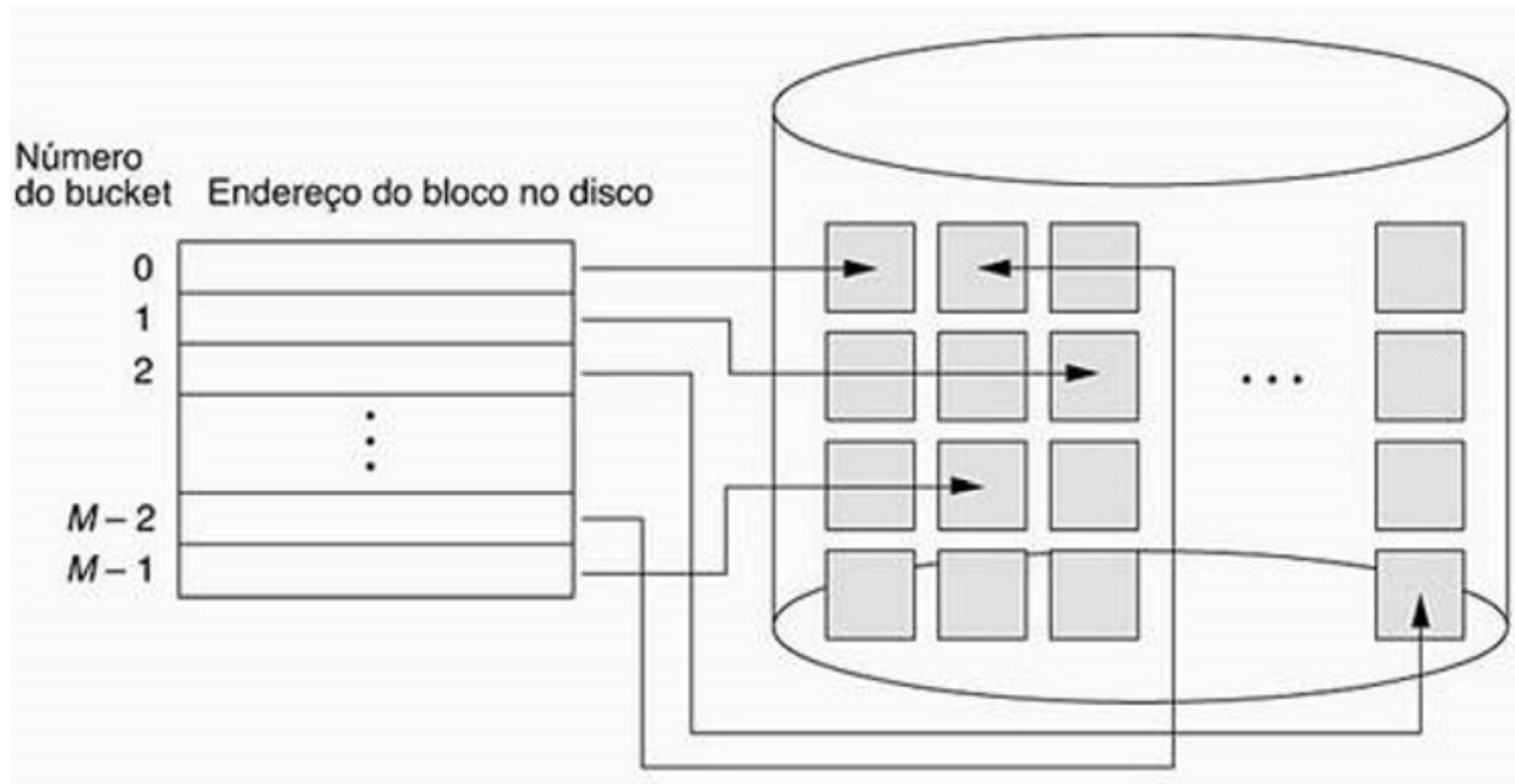
- ◆ (núm. do agrupamento, endereço do bloco no arquivo)

- ◆ Uma tabela mantida no cabeçalho do arquivo converte o número do agrupamento para o endereço do bloco de disco correspondente

→ Abertura de um agrupamento de *overflow* que é apontado pelo agrupamento original

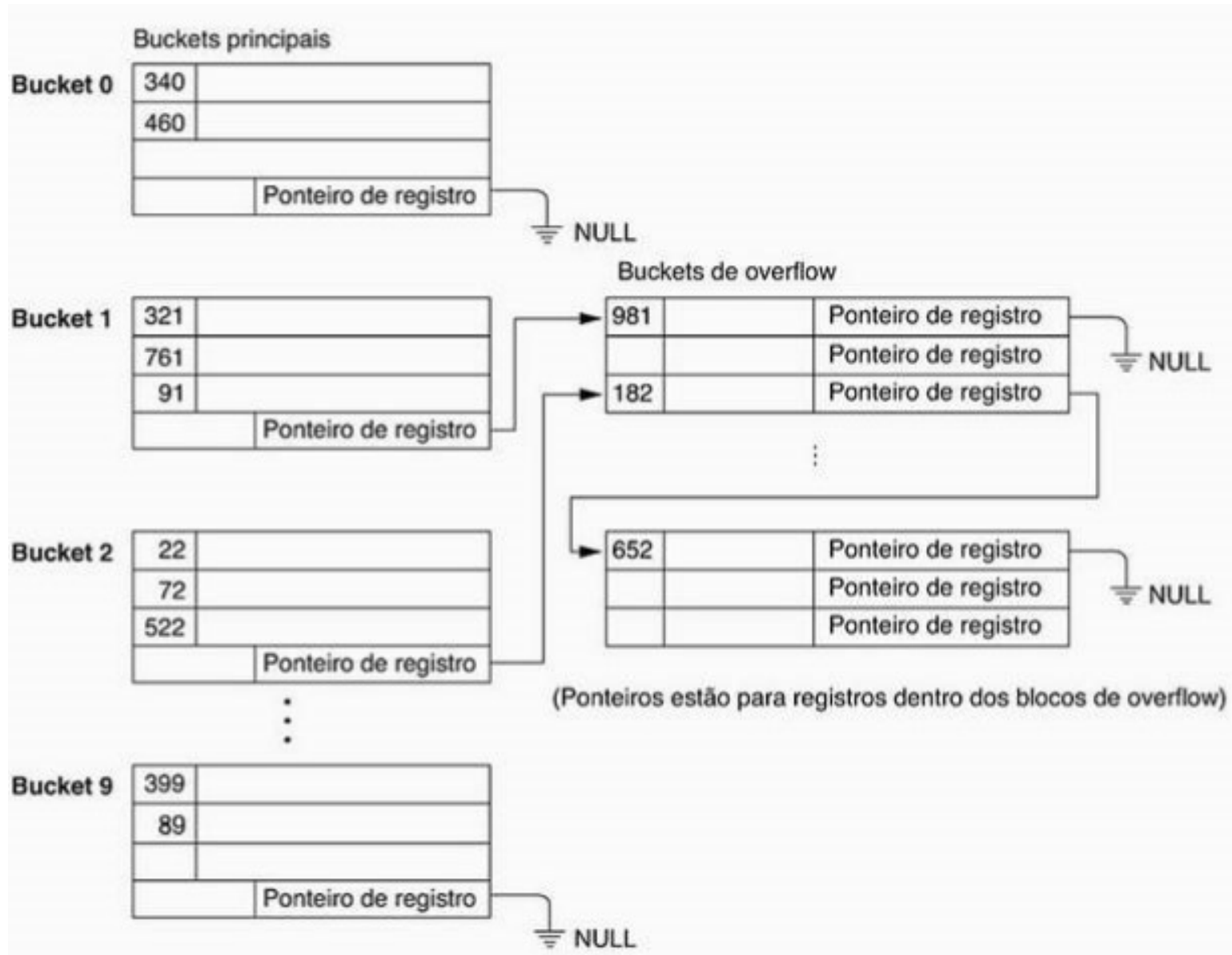
→ **Sério:** a busca por outro campo que não seja o campo de *hashing* ⁵³

Hashing Externo



→ Correspondendo números de *bucket* a endereços de blocos de disco

Hashing Externo



Hashing Externo

→ Até agora estático

- Quantidade fixa de buckets M alocados
- Tamanho grande de um bucket para poucos registros
- Tamanho pequeno de um bucket para muitos registros

→ Dinâmico

- ...

→ Outras formas de organização

- Arquivos de registros mistos
- B-trees e outras estruturas de dados como organização primária

→ Outras formas de armazenamento

- RAID
- Redes de áreas de armazenamento
- Armazenamento conectado à rede
- iSCSI